

Playwright Core: Setup, Locators & First Tests

Session 3 · Playwright Automation Bootcamp

TESTMART

LOCATORS

AUTO-WAIT

FIRST JOURNEY



What You Will Be Able to Do

01

Install & Configure

Set up Playwright for TestMart with correct baseURL

02

Understand Lifecycle

Explain browser, context, and page hierarchy

03

Master Locators

Use role, label, text, testid, CSS, and XPath correctly

04

Handle Edge Cases

Strict mode violations, auto-wait, spinner pattern, negative assertions

05

Full Journey

Complete Login → Search → Logout independently



Auto-Wait in Practice: The Spinner Pattern

WAIT FOR CONDITIONS, NEVER FOR TIME

❌ Wrong — arbitrary wait

```
page.getByTestId('search-  
btn').click()  
page.waitForTimeout(3000) // brittle!  
expect(getByTestId('product-card'))  
  .toHaveCount(1)
```

✅ Correct — assert the condition

```
page.getByTestId('search-btn').click()  
expect(getByTestId('loading-spinner'))  
  .toBeVisible() // fetch started  
expect(getByTestId('loading-spinner'))  
  .not.toBeVisible() // fetch done  
expect(getByTestId('product-card'))  
  .toHaveCount(1) // safe to assert
```



If you reach for `waitForTimeout`, you haven't identified what you're waiting for. **Find the condition and assert it explicitly.**

Counting Elements with `toHaveCount`

ASSERT ON LISTS OF ELEMENTS

Home Page

```
expect(getByTestId('product-card'))  
  .toHaveCount(4)
```

4 featured products shown

After Category Filter

```
page.getByTestId('filter-electronics').click()  
expect(getByTestId('product-card'))  
  .toHaveCount(4) // auto-retries
```

After Search

```
expect(getByTestId('loading-spinner'))  
  .not.toBeVisible() // wait for fetch  
  
expect(getByTestId('product-card'))  
  .toHaveCount(1)
```



`toHaveCount` **auto-retries** — no need to add extra waits.

Negative Assertions

ASSERT WHAT SHOULD NOT BE THERE

TestMart Auth State

```
// Before login
expect(getByTestId('nav-login'))
  .toBeVisible()
expect(getByTestId('nav-logout'))
  .not.toBeVisible()

// After login (state flips)
expect(getByTestId('nav-logout'))
  .toBeVisible()
expect(getByTestId('nav-login'))
  .not.toBeVisible()
```

`not.toBeVisible()` vs `not.toBeAttached()`

Assertion	Passes when...	Use
<code>not.toBeVisible()</code>	Element hidden OR absent from DOM	Default — resilient
<code>not.toBeAttached()</code>	Element completely absent from DOM	Strict — use sparingly



Always default to `not.toBeVisible()` — resilient to implementation changes.

Lab 1 — Locator Playground

45 MINUTES

Six focused exercises — one concept each, no full flows. Inline hints available when stuck.

1

`toHaveTitle`, `toBeVisible`

Assert home page elements

2

`getByRole`

Button vs link distinction on home page

3

`toHaveCount`

Count 4 on home, 12 on products page

4

Nested Locators

Strict mode — scope with `.first()`

5

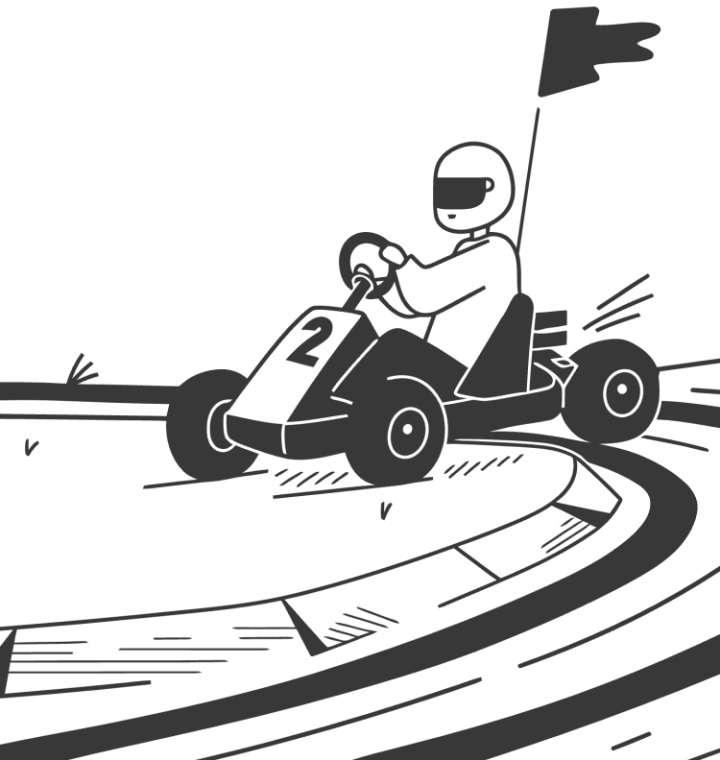
Three Strategies

Same email input, three locator approaches

6

`not.toBeVisible()`

Nav state before and after login



Full Journey: Login → Search → Logout

Put it all together — a complete end-to-end flow using the locator strategies from this session.

1

Login

`getByLabel` for email & password, `getByRole('button')` to submit

2

Search

`getByPlaceholder` to fill query, `getByTestId('loading-spinner')` for async wait

3

Assert Results

`getByText` for product names, `not.toBeVisible()` to confirm spinner gone

4

Logout

`getByRole('button', { name: 'Logout' })` — verify redirect with `toHaveURL`



Complete this flow independently

Lab 1 — First Full Journey

45 MINUTES

Guided: Login → Search → Logout

01

Navigate home, assert `nav-login` visible

02

Click login, fill credentials, submit

03

Assert `nav-logout` visible, username shows **Alex**

04

Search "Keyboard" using the spinner wait pattern

05

Assert 1 result: **Mechanical Keyboard**

06

Logout, assert back on home

Independent Exercise

- Filter by Electronics
- Navigate to product detail
- Attempt unauthenticated cart click
- Get redirected to login
- Log in and verify redirect



Hints and solution available — no guidance provided upfront.

Key Takeaways



Locator Priority

Prefer role/text/testid. Use CSS/XPath only as a last resort.



Strict Mode

Scope locators to one element. Ambiguity is a test bug.



Auto-Wait

Built into every action. Never use `waitForTimeout`.



Negative Assertions

Default to `not.toBeVisible()` for resilient absence checks.

